ET 2.0 Depth-First Branch-and-Bound

Extra Topics for Unit 2: Heuristic Search

Big idea: keep the cheapest solution found so far. Use it to prune branches whose estimated cost is already too high.

Today

- the bound and pruning
- two worked examples
- guarantees and costs

The Idea

What the search does

Depth-first branch-and-bound (**DFBnB**) searches for the **cheapest path to a goal**. It follows one branch at a time, as depth-first search does. After finding a solution, it keeps searching for a cheaper one.

What it remembers

- The **best path to a goal** found so far. This is the *incumbent solution*.
- **UB**, the cost of that path. Before any solution is found, UB $= \infty$.

What it prunes

If a node's estimated total cost exceeds UB, the search **prunes** that node. It skips the node and all paths that extend it. A safe estimate never overestimates the cheapest such solution.

What g, h, and f Mean

A node represents one particular path from the start

$g(n)$	The cost of that path so far.
$h(n)$	An estimate of the cheapest remaining cost from its state to any goal.
$f(n)$	$g(n) + h(n)$: the estimated total cost of a solution extending that path.

The same state can appear in different tree nodes

In Example 1, S→B costs 10, while S→A→B costs 8. Both end at B and use $h(\mathrm{B}) = 2$. Their f values are 12 and 10.

We use nonnegative edge costs and $h \geq 0$, with $h = 0$ at every goal. At a goal, $f = g$ is the actual solution cost.

Why Pruning Is Safe

Admissible means the remaining-cost estimate is never too high

If h is **admissible**, $f(n) = g(n) + h(n)$ is a **lower bound** on the cost of every solution extending node n. After a solution is found, UB is an **upper bound** on the optimal cost: we already have a solution that costs UB.

Example: UB $= 15$ and $f(n) = 17$

Every solution extending n costs at least 17. We already have one that costs 15. We can safely skip the whole branch below n.

This deck prunes when $f > \mathrm{UB}$ and keeps $f = \mathrm{UB}$. Using $f \geq \mathrm{UB}$ is also safe when only one cheapest solution is needed. Use one convention consistently.

The Order Is Still Depth First

Choose a group first, then a node within it

1. A **sibling group** contains the unprocessed children of one expanded node.
2. Always use the **newest nonempty group**. Choose its lowest-f child. Break ties by generation order, from left to right.
3. Expanding that child creates a newer group. Finish that group and its descendants before returning to the waiting siblings above it.

Example 1: a waiting node can have a smaller f

A waits with $f = 13$. After B is expanded, its children form the newest group. DFBnB pops G2 with $f = 15$ before A. A* would choose the smallest f across the whole frontier.

The Steps: Start and Choose a Node

Initialize once

Set UB $= \infty$ and the best path to *none*. Put the start node, with $g = 0$, on the **frontier**: the nodes waiting to be processed. It forms a group of its own.

Repeat until the frontier is empty

1. Remove empty groups until the newest group is nonempty.
2. **Pop** its lowest-f node: remove that node from the frontier.
3. Process the popped node using the rules on the next slide. Then repeat.

When the frontier is empty, return the saved best path. If no goal was found, report *no solution*.

The Steps: Process the Popped Node

Apply these cases in order

1. **If $f(n) > \mathrm{UB}$: prune n.** Do not expand it.
2. **Otherwise, if n is a goal:** if $g(n) < \mathrm{UB}$, save its path and set $\mathrm{UB} = g(n)$. If it ties UB, keep the saved path. **After processing a goal, backtrack; do not generate its children.**
3. **Otherwise, expand n.** Generate its children from left to right. Discard each child with $f > \mathrm{UB}$ immediately. The children that remain form the newest sibling group.

Why check the bound both at generation and at pop?

A child may enter the frontier while UB is large, then wait while a cheaper solution lowers UB. Check against the **current** UB when the child is popped.

How to Read the Worked Traces

The table records the search as it runs

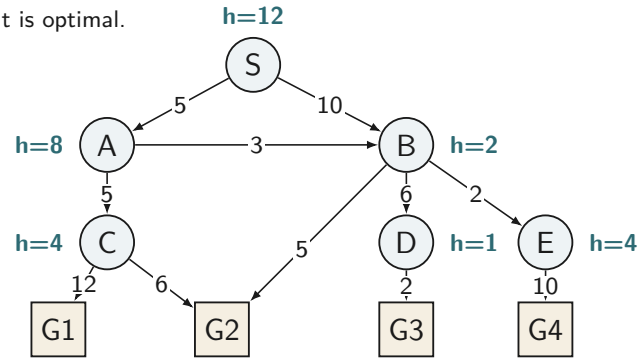
ID / pop	ID identifies a column on this slide. Pop gives the removal order.
Dark text	A node waiting on the frontier, or the node being processed now.
Magenta	The highlighted node is being popped.
Gray text	A processed node, kept in the table as a record.
Red text / cross	A pruned node. A blank pop entry means it was discarded at generation.

Arrows show directed edges and their costs. Squares mark goals; each has $h = 0$. A green check marks a recorded goal. This is **tree search**: reaching a state by another path creates a new node. The displayed history is not all stored by the search.

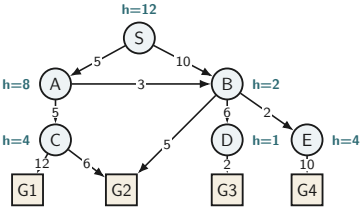
Example 1: Run DFBnB from S

Run **depth-first branch-and-bound** (tree search) from **S** to find the cheapest path to a goal. Track **UB** and the best path at every step.

- Give the final path.
- Give its cost.
- Explain whether it is optimal.



DFBnB Example 1: First Descent



Expansion tree

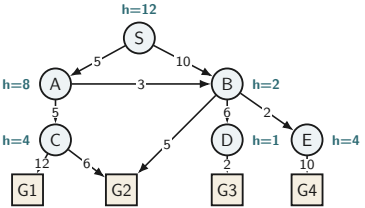
Frontier:

pop:						
ID:	0	1	2	3	4	5
state:						
parent:						
g:						
h:						
$f = g + h$:						

UB: ∞
Best path:

▶ **The rules.** UB starts at ∞. Pop the lowest f of the newest sibling group. Prune if $f >$ UB.

DFBnB Example 1: First Descent



Expansion tree



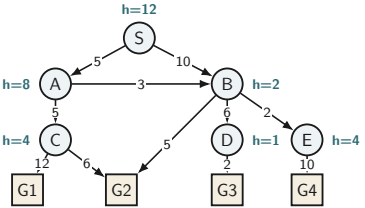
Frontier:

pop:						
ID:	0	1	2	3	4	5
state:	S					
parent:	–					
g:	0					
h:	12					
$f=g+h$:	12					

UB: ∞
Best path:

▶ **Initialize.** Push the start node **S** with g=0 and f=0+12=12.

DFBnB Example 1: First Descent



Expansion tree



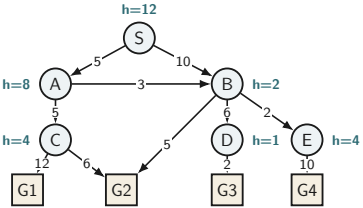
Frontier:

pop:	**1**					
ID:	0	1	2	3	4	5
state:	S					
parent:	–					
g:	0					
h:	12					
$f = g + h$:	12					

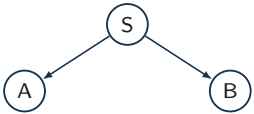
UB: ∞
Best path:

▶ **Pop S**, the only node on the frontier.

DFBnB Example 1: First Descent



Expansion tree



Frontier:

pop:	1					
ID:	0	1	2	3	4	5
state:	S	A	B			
parent:	–	S	S			
g:	0	5	10			
h:	12	8	2			
$f=g+h$:	12	13	12			

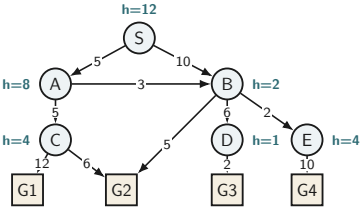
siblings

UB: ∞

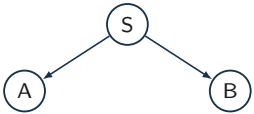
Best path:

▶ **Expand S.** Generate **A** (f=13), then **B** (f=12). Both have $f \leq$ UB, so both are added.

DFBnB Example 1: First Descent



Expansion tree



Frontier:

pop:	1		2			
ID:	0	1	2	3	4	5
state:	S	A	B			
parent:	–	S	S			
g:	0	5	10			
h:	12	8	2			
$f = g + h$:	12	13	12			

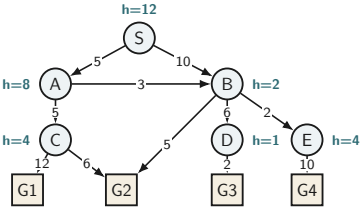
siblings

UB: ∞

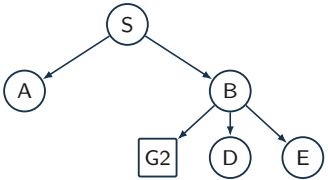
Best path:

▶ **Pop B**, the lowest f (12) of the newest sibling group {A, B}.

DFBnB Example 1: First Descent



Expansion tree



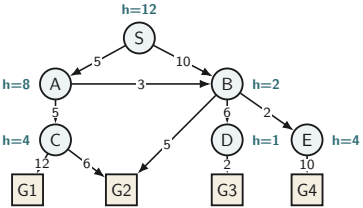
Frontier:

pop:	1		2			
ID:	0	1	2	3	4	5
state:	S	A	B	G2	D	E
parent:	–	S	S	B	B	B
g:	0	5	10	15	16	12
h:	12	8	2	0	1	4
$f=g+h$:	12	13	12	15	17	16
		siblings		*siblings*		

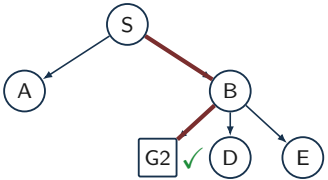
UB: ∞
Best path:

▶ **Expand B.** Generate **G2** (f=15), **D** (f=17), **E** (f=16). All have $f \leq$ UB, so all are added.

DFBnB Example 1: First Descent



Expansion tree



Frontier:

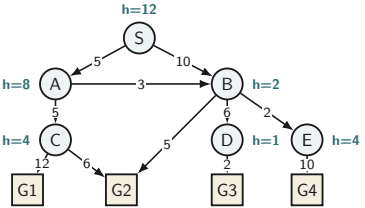
pop:	1		2	**3**		
ID:	0	1	2	3	4	5
state:	S	A	B	G2	D	E
parent:	–	S	S	B	B	B
g:	0	5	10	15	16	12
h:	12	8	2	0	1	4
$f=g+h$:	12	13	12	15	17	16

siblings *siblings*

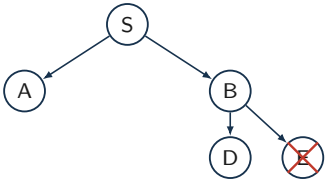
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Pop G2** (f=15). It is a **goal**. $15 < \infty$, so **UB** = 15 and the best path is S→B→G2.

DFBnB Example 1: First Descent



Expansion tree



Frontier:

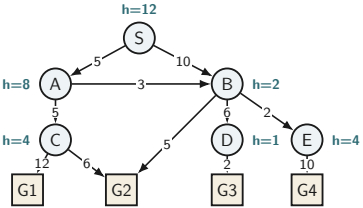
pop:	1		2	3		4
ID:	0	1	2	3	4	5
state:	S	A	B	G2	D	E
parent:	–	S	S	B	B	B
g:	0	5	10	15	16	12
h:	12	8	2	0	1	4
$f=g+h$:	12	13	12	15	17	16

siblings *siblings*

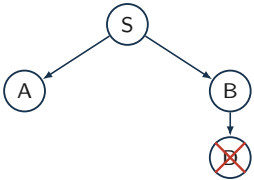
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Pop E** (f=16). 16 > UB = 15, so E is **pruned**.

DFBnB Example 1: First Descent



Expansion tree



Frontier:

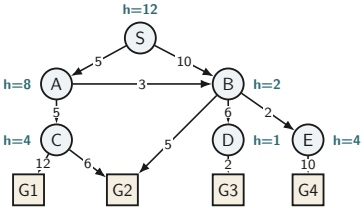
pop:	1		2	3	**5**	**4**
ID:	0	1	2	3	4	5
state:	S	A	B	G2	D	E
parent:	–	S	S	B	B	B
g:	0	5	10	15	16	12
h:	12	8	2	0	1	4
$f=g+h$:	12	13	12	15	17	16

siblings *siblings*

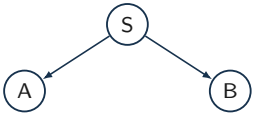
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Pop D** (f=17). 17 > 15, so D is **pruned**. The sibling group of B's children is now empty.

DFBnB Example 1: First Descent



Expansion tree



Frontier:

pop:	1		2	3	**5**	**4**
ID:	0	1	2	3	4	5
state:	S	A	B	G2	D	E
parent:	–	S	S	B	B	B
g:	0	5	10	15	16	12
h:	12	8	2	0	1	4
$f=g+h$:	12	13	12	15	17	16

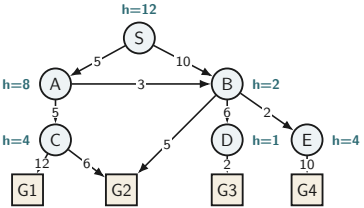
siblings *siblings*

UB: 15

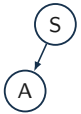
Best path: S → B → G2 (cost 15)

▶ **Backtrack** to S's child group. Only **A** (f=13) remains. Continue from A with UB = 15.

DFBnB Example 1: Second Descent



Expansion tree



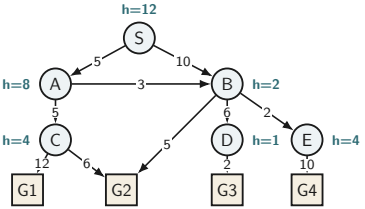
Frontier:

pop:						
ID:	0	1	2	3	4	5
state:	A					
parent:	S					
g:	5					
h:	8					
$f=g+h$:	13					

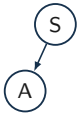
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Second descent.** UB stays 15. **A** (f=13), the other child of S, is still on the frontier.

DFBnB Example 1: Second Descent



Expansion tree



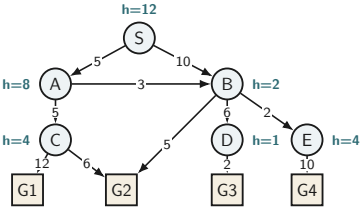
Frontier:

pop:	**6**					
ID:	0	1	2	3	4	5
state:	A					
parent:	S					
g:	5					
h:	8					
$f=g+h$:	13					

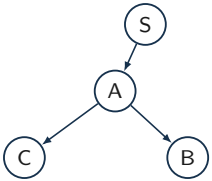
UB: 15
Best path: S → B → G2 (cost 15)

- **Pop A** (f=13). 13 ≤ UB = 15, so A is not pruned. A is not a goal, so it is expanded.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

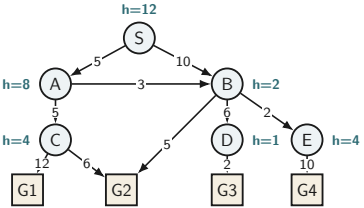
pop:	6					
ID:	0	1	2	3	4	5
state:	A	C	B			
parent:	S	A	A			
g:	5	10	8			
h:	8	4	2			
$f=g+h$:	13	14	10			

siblings

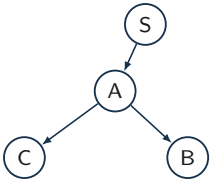
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Expand A.** Generate **C** (f=14), then **B** (f=10). Both have $f \leq$ UB, so both are added.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

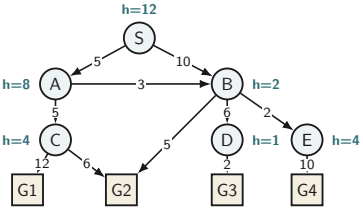
pop:	6		7			
ID:	0	1	2	3	4	5
state:	A	C	B			
parent:	S	A	A			
g:	5	10	8			
h:	8	4	2			
$f=g+h$:	13	14	10			

siblings

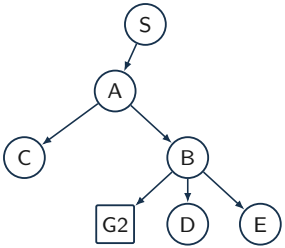
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Pop B via A** (f=10), before C (f=14). This is a new path to B: g=8, instead of 10.

DFBnB Example 1: Second Descent



Expansion tree



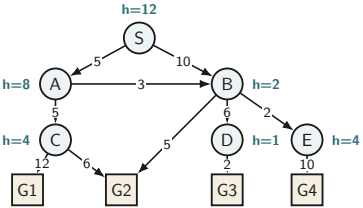
Frontier:

pop:	6		7			
ID:	0	1	2	3	4	5
state:	A	C	B	G2	D	E
parent:	S	A	A	B	B	B
g:	5	10	8	13	14	10
h:	8	4	2	0	1	4
$f=g+h$:	13	14	10	13	15	14
		siblings		*siblings*		

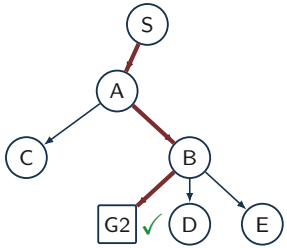
UB: 15
Best path: S → B → G2 (cost 15)

▶ **Expand B.** Generate **G2** (f=13), **D** (f=15), **E** (f=14). None has $f >$ UB, so all are added.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

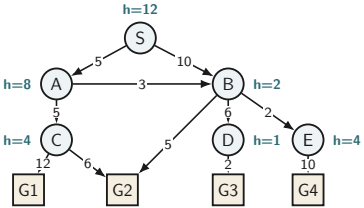
pop:	6		7	8		
ID:	0	1	2	3	4	5
state:	A	C	B	G2	D	E
parent:	S	A	A	B	B	B
g:	5	10	8	13	14	10
h:	8	4	2	0	1	4
$f=g+h$:	13	14	10	13	15	14

siblings *siblings*

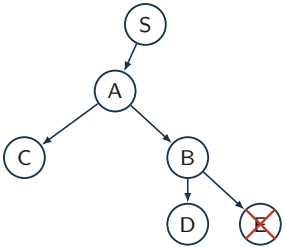
UB: 13
Best path: S → A → B → G2 (cost 13)

▶ **Pop G2** (f=13). It is a **goal**. 13 < 15, so **UB** = 13 and the best path is S→A→B→G2.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

pop:	6		7	8		9
ID:	0	1	2	3	4	5
state:	A	C	B	G2	D	E
parent:	S	A	A	B	B	B
g:	5	10	8	13	14	10
h:	8	4	2	0	1	4
$f=g+h$:	13	14	10	13	15	14

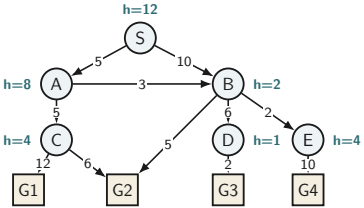
siblings *siblings*

UB: 13

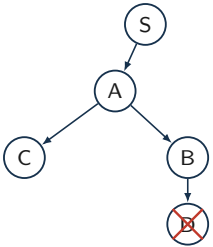
Best path: S → A → B → G2 (cost 13)

▶ **Pop E** (f=14). 14 > UB = 13, so E is **pruned**.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

pop:	6		7	8	**10**	9
ID:	0	1	2	3	4	5
state:	A	C	B	G2	D	E
parent:	S	A	A	B	B	B
g:	5	10	8	13	14	10
h:	8	4	2	0	1	4
$f=g+h$:	13	14	10	13	15	14

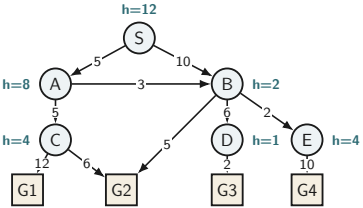
siblings *siblings*

UB: 13

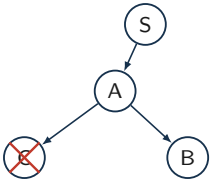
Best path: S → A → B → G2 (cost 13)

▶ **Pop D** (f=15). 15 > 13, so D is **pruned**. The sibling group of B's children is now empty.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

pop:	6	**11**	7	8	**10**	**9**
ID:	0	1	2	3	4	5
state:	A	C	B	G2	D	E
parent:	S	A	A	B	B	B
g:	5	10	8	13	14	10
h:	8	4	2	0	1	4
$f=g+h$:	13	14	10	13	15	14

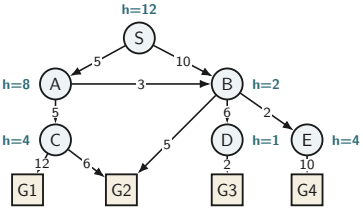
siblings *siblings*

UB: 13

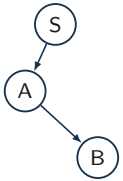
Best path: S → A → B → G2 (cost 13)

▶ **Backtrack** to A's child group. Only **C** remains (f=14). 14 > 13, so C is **pruned**.

DFBnB Example 1: Second Descent



Expansion tree



Frontier:

pop:	**6**	**11**	**7**	**8**	**10**	**9**
ID:	0	1	2	3	4	5
state:	A	C	B	G2	D	E
parent:	S	A	A	B	B	B
g:	5	10	8	13	14	10
h:	8	4	2	0	1	4
$f = g + h$:	13	14	10	13	15	14

siblings *siblings*

UB: 13

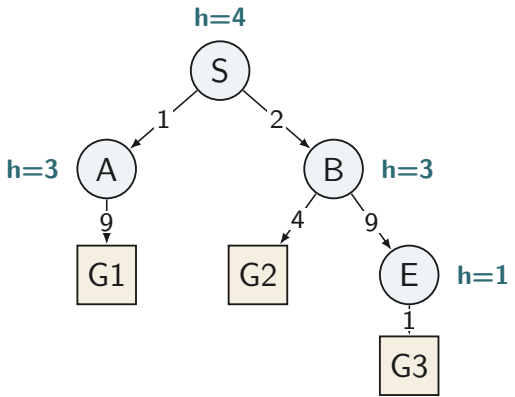
Best path: S → A → B → G2 (cost 13)

✓ **optimal: cost 13**

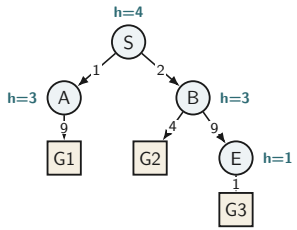
▶ The frontier is empty. **Return** S→A→B→G2, cost **13**. With admissible h, no cheaper path was pruned.

Example 2: A Prune at Generation

Run DFBnB from **S** on this second graph. Here one child is **pruned when it is generated**, because its $f = g + h$ is already greater than UB.



DFBnB Example 2



Expansion tree

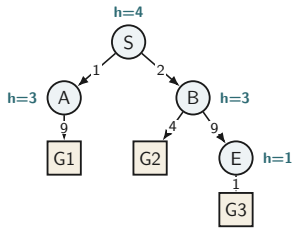
Frontier:

pop:						
ID:	0	1	2	3	4	5
state:						
parent:						
g:						
h:						
$f = g + h$:						

UB: ∞
Best path:

▶ **Example 2.** The rules are the same. One child will be pruned *at generation*.

DFBnB Example 2



Expansion tree



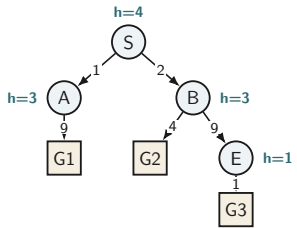
Frontier:

pop:						
ID:	0	1	2	3	4	5
state:	S					
parent:	–					
g:	0					
h:	4					
$f=g+h$:	4					

UB: ∞
Best path:

▶ **Initialize.** Push the start node **S** with g=0 and f=0+4=4. UB = ∞.

DFBnB Example 2



Expansion tree



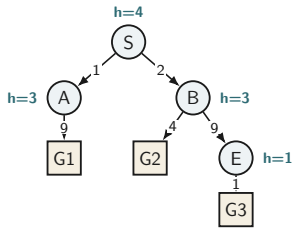
Frontier:

pop:	**1**					
ID:	0	1	2	3	4	5
state:	S					
parent:	–					
g:	0					
h:	4					
$f = g + h$:	4					

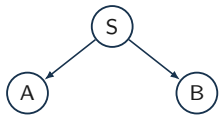
UB: ∞
Best path:

▶ **Pop S**, the only node on the frontier.

DFBnB Example 2



Expansion tree



Frontier:

	pop:	1					
	ID:	0	1	2	3	4	5
	state:	S	A	B			
	parent:	–	S	S			
g:		0	1	2			
h:		4	3	3			
$f=g+h$:		4	4	5			

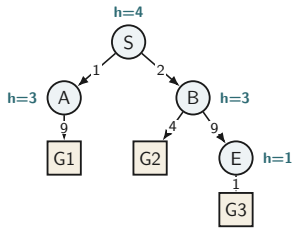
siblings

UB: ∞

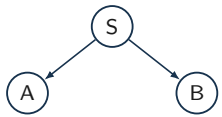
Best path:

▶ **Expand S.** Generate **A** (f=4), then **B** (f=5). Both have $f \leq$ UB, so both are added.

DFBnB Example 2



Expansion tree



Frontier:

pop:	1	**2**				
ID:	0	1	2	3	4	5
state:	S	A	B			
parent:	–	S	S			
g:	0	1	2			
h:	4	3	3			
$f = g + h$:	4	4	5			

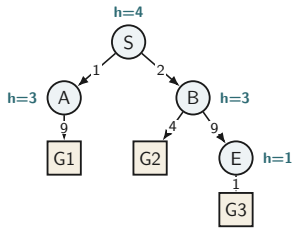
siblings

UB: ∞

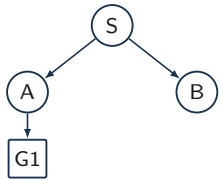
Best path:

▶ **Pop A**, the lowest f (4) of the newest sibling group {A, B}.

DFBnB Example 2



Expansion tree



Frontier:

pop:	1	2				
ID:	0	1	2	3	4	5
state:	S	A	B	G1		
parent:	–	S	S	A		
g:	0	1	2	10		
h:	4	3	3	0		
$f=g+h$:	4	4	5	10		

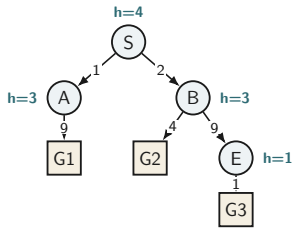
siblings

UB: ∞

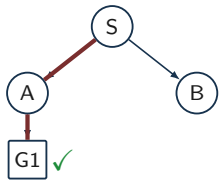
Best path:

▶ **Expand A.** Generate **G1** with g=10 and f=10. 10 $\leq$ UB, so G1 is added.

DFBnB Example 2



Expansion tree



Frontier:

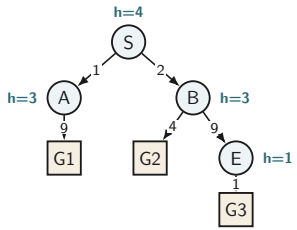
pop:	1	2		3		
ID:	0	1	2	3	4	5
state:	S	A	B	G1		
parent:	–	S	S	A		
g:	0	1	2	10		
h:	4	3	3	0		
$f=g+h$:	4	4	5	10		

siblings

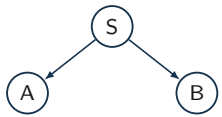
UB: 10
Best path: S → A → G1 (cost 10)

▶ **Pop G1** (f=10). It is a **goal**. $10 < \infty$, so **UB** = 10 and the best path is S→A→G1.

DFBnB Example 2



Expansion tree



Frontier:

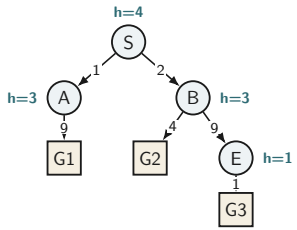
pop:	1	2	**4**	3		
ID:	0	1	2	3	4	5
state:	S	A	B	G1		
parent:	–	S	S	A		
g:	0	1	2	10		
h:	4	3	3	0		
$f=g+h$:	4	4	5	10		

siblings

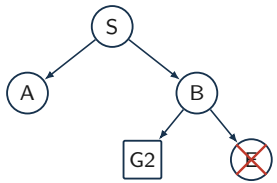
UB: 10
Best path: S → A → G1 (cost 10)

▶ **Backtrack** to S's child group and **pop B** (f=5). $5 \leq \text{UB} = 10$, so B is expanded.

DFBnB Example 2



Expansion tree



Frontier:

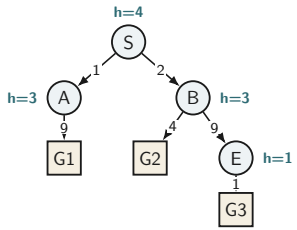
pop:	1	2	4	3		
ID:	0	1	2	3	4	5
state:	S	A	B	G1	G2	E
parent:	–	S	S	A	B	B
g:	0	1	2	10	6	11
h:	4	3	3	0	0	1
$f=g+h$:	4	4	5	10	6	12
		siblings			*siblings*	

E never enters the frontier

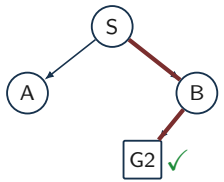
UB: 10
Best path: S → A → G1 (cost 10)

▶ **Expand B. G2** (f=6) is added. **E** has f=12 > UB = 10, so E is **pruned at generation**.

DFBnB Example 2



Expansion tree



Frontier:

pop:	1	2	4	3	5	
ID:	0	1	2	3	4	5
state:	S	A	B	G1	G2	E
parent:	–	S	S	A	B	B
g:	0	1	2	10	6	11
h:	4	3	3	0	0	1
$f=g+h$:	4	4	5	10	6	12

siblings *siblings*

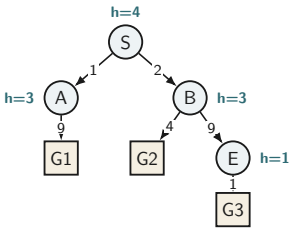
E never enters the frontier

UB: 6

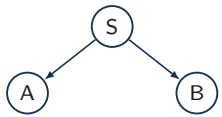
Best path: S → B → G2 (cost 6)

▶ **Pop G2** (f=6). It is a **goal**. $6 < 10$, so **UB** = 6 and the best path is S→B→G2.

DFBnB Example 2



Expansion tree



Frontier:

pop:	1	2	4	3	5	
ID:	0	1	2	3	4	5
state:	S	A	B	G1	G2	E
parent:	–	S	S	A	B	B
g:	0	1	2	10	6	11
h:	4	3	3	0	0	1
$f=g+h$:	4	4	5	10	6	12

siblings *siblings*

E never enters the frontier

UB: 6

Best path: S → B → G2 (cost 6)

✓ **optimal: cost 6**

▶ The frontier is empty. **Return** S→B→G2 with cost **6**. The first path found, to G1, cost 10.

Properties: What DFBnB Guarantees

A finite search tree and an admissible heuristic

With these conditions, the search terminates and returns a cheapest solution if one exists. If no solution exists, it reports failure. Both example graphs are finite and have no cycles, so their search trees are finite.

Why the final saved solution is optimal

1. The search finishes every branch it does not prune.
2. A pruned branch cannot improve the solution saved at the time of pruning. Later saved solutions can only be cheaper.
3. When the frontier is empty, no unexplored branch can contain a cheaper solution.

Stopping Early and Handling Cycles

Anytime: a usable solution after the first goal is found

You can stop and return the best solution found so far. Before the first goal, there is **no solution to return**. Stopping early does not by itself prove optimality. Example 2 first finds a solution of cost 10; finishing the search improves it to 6.

A finite graph can still have an infinite search tree

- With UB $= \infty$, tree search can follow a cycle forever before finding a goal.
- For a finite graph with nonnegative costs, reject a child whose state already occurs on its **current path**. Removing a cycle cannot increase path cost.
- This path check preserves a cheapest solution. A global "visited once" rule can lose a cheaper path to a state, such as the second path to B in Example 1.

Properties: Memory

Keep the current path and its waiting siblings

Let b be the maximum number of children per node and m the maximum search depth. At most one sibling group per depth is active. With parent links and discarded nodes freed, the frontier and path records take $O(bm)$ space. The saved best path takes $O(m)$.

Linear in depth when the branching factor is fixed

Example 1 generates 11 tree nodes in total, but at most 4 wait on the frontier at once. The tables keep old entries so you can follow the trace; the search need not keep that history.

A* keeps candidates from many branches at once. Its frontier can grow exponentially with search depth.

Properties: Time

A small frontier does not mean little search

- Different paths to a state are searched separately. Example 1 expands B twice.
- Before UB becomes small, DFBnB may explore expensive branches. Example 2 pops G1 with $f = 10$ before finding the solution of cost 6. With admissible h, A* tree search selects no node with f greater than the optimal cost before returning a goal.
- In the worst case, pruning removes very little of the search tree. For fixed $b > 1$ and maximum depth m, the number of nodes processed can be $O(b^m)$.

Trying promising children first can find a cheap solution sooner. A smaller UB then permits more pruning. Child order affects efficiency, not the optimality guarantee on a finite tree.

Admissible Does Not Mean Consistent

Example 1 uses an admissible heuristic

State	S	A	B	C	D	E
h	12	8	2	4	1	4
True cheapest remaining cost	13	8	5	6	2	10

Every estimate is at most the true remaining cost. Each goal has both values equal to 0.

Yet f can decrease along a path

A consistent heuristic cannot drop by more than the cost of an edge. For A$\rightarrow$B, this requires $h(\mathrm{A}) \leq 3 + h(\mathrm{B})$, but $8 > 3 + 2$. Along S$\rightarrow$A$\rightarrow$B, f drops from 13 at A to 10 at B. DFBnB's pruning remains safe: it needs admissibility, not consistency.

Takeaways

- **The bound.** UB is the cost of the best solution found so far. It starts at ∞ and it only decreases.
- **The rule.** Prune a node when $f = g + h >$ UB, at generation and again at pop.
- **The order.** Depth-first, and the lowest f of the newest sibling group first.
- **The guarantee.** With admissible h, a solution returned after the frontier is empty is optimal. A finite search tree ensures termination.
- **The cost.** Space is $O(bm)$; time can be exponential in depth.

Class Question

In Example 1, change $h(\mathrm{A})$ from 8 to 11 and keep every other value.

After the first goal, which waiting node is pruned? What path is returned?